

Jadavpur University



Computer Networks **Lab Report**

NAME : ANKIT SHAW

ROLL NO. : 002310501032

YEAR : BCSE UG – III (A1)

SESSION : 2023 – 27

ASSIGNMENT : 0 2

Title: -

Problem Statement :

Implement flow control mechanisms for Data Link Layer.

Sender Program: The Sender program consist of following methods:

Framing(): This method will prepare the frame following the structure given below. In the header section, the MAC address of the source and destination are specified. Payload is the data of fixed size (pre decided value within the range 46-1500 bytes e.g., 46 bytes) from the input text file. Frame check Sequence using CRC/Checksum (using the CRC/Checksum module of assignment 1) is appended as a trailer.

Channel(): The channel method introduces random delay (this will cause packet loss or timeout) and/or bit error (using the error injection module of assignment 1) while transferring frames.

Send(): Sender program will send/transmit data frame using socket connection to Receiver program. Sender should decide whether to send a new data frame or retransmit the same frame again due to timeout.

Timer(): Timer will be associated with each frame transmission. It will be used to check the timeout condition.

Timeout(): This function should be called to compute the most recent data frame's round-trip time and then re-compute the value of timeout.

Recv(): This method is invoked by the sender program whenever an ACK packet is received. Need to consider network time when the ACK was received to check the timeout condition.

Receiver Program: Receiver program consist of following methods:

Recv(): This method is invoked by the Receiver program whenever a Data frame is received.

Check(): This method checks (using CRC/Checksum of assignment 1) if there is any error in data. The data frame is discarded if an error is detected otherwise accepted.

Send(): Receiver program will prepare an acknowledgement frame and send it to the sender as a response to successful receipt of the data frame.

Flow Control: Implement the following flow control approaches using the Sender and Receiver program and their methods:

Stop and Wait: The Sender program calls Send() method to transmit one frame at a time slot to the receiver. After that Sender will wait for the ACK frame from the receiver as a response to a successful receipt of the data frame. The interval between sending a message and receiving an acknowledgment is known as the sender's waiting time, and the sender is idle during this time. After receiving an ACK, the sender will send the next data packet to the receiver, and so on, as long as the sender has data to send.

Go-Back-N ARQ: The Send() method of the Sender program will take N as input. The Sender's window size is N. So that the sender can send N frames which are equal to the window size. The size of the receiver's window is 1. Once the entire window is sent, the sender then waits for a cumulative acknowledgement to send more packets. That is, receiving acknowledgment for frame i means the frames i-1, i-2, and so on are acknowledged as well. You can specify that acknowledgement no. in the ACK frame. Receiver receives only in-order packets and discards out-of-order frames and corrupted frames (checked by CRC or Checksum). In case of packet loss, the entire window would be re-transmitted.

Selective Repeat ARQ: The Send() method of the Sender program will take N as input. The Sender's window size and Receiver's window size both are N. The sender sends N frames and the receiver acknowledges all frames whether they were received in order or not. Remember that type of acknowledgement here is not cumulative. It uses Independent Acknowledgement to acknowledge the packets. In this case, the receiver maintains a buffer to contain out-of-order packets and sorts them. Receiver discards only corrupted frames (checked by CRC or Checksum)

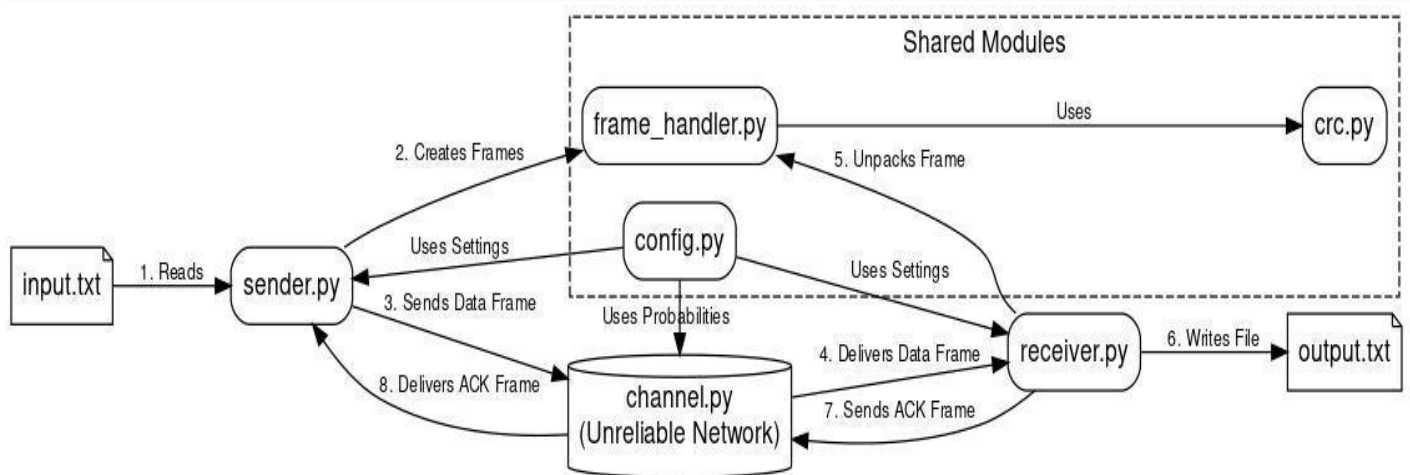
Test the above three schemes for the following cases (not limited to):

1. Compare time between the propagation of a packet and reception of its ACK.
2. Compare efficiency of the above approaches without error or lost frame.
3. Compare efficiency of the above approaches for different probability (0.1 - 0.5) of an error or delay in the transmission of a packet or in its acknowledgment.

Design

This program's purpose is to simulate and compare different data link layer flow control protocols, namely Stop-and-Wait, Go-Back-N, and Selective Repeat ARQ. It achieves this by sending a file from a sender to a receiver over a simulated network environment where adverse conditions like packet loss, bit errors, and transmission delays can be introduced and controlled. By running these simulations, the program allows for a practical comparison of each protocol's efficiency, reliability, and performance, ultimately generating an efficiency report with metrics like throughput and the total number of retransmissions required for a successful transfer.

Schematic Diagram :



Input & Output Formats:

The primary input is an `input.txt` file, which can be any binary file, and the primary output is an `output.txt` file, which is a perfect copy of the input. The program also takes user selections and settings from `config.py` as input and produces detailed terminal logs and an efficiency report as output.

Input Format:

1. Data File (`input.txt`)

Format: Any standard file. The program reads it in binary mode ("rb"), so it can be a text file, an image, an executable, or any other file type.

Purpose: This is the data payload that the simulation transmits from the sender to the receiver to test the flow control protocols.

2. Configuration File (config.py)

Format: A Python script containing constant variables.

Purpose: This file allows you to control all simulation parameters without changing the main code. This includes network addresses, ports, payload size, window size, and the probabilities for packet loss, bit errors, and delays.

3. User Input (Terminal)

Format: Numerical choices(1, 2, 3, 4)entered into the command-line menu.

Purpose: To allow the user to select which flow control protocol to simulate or to quit the program.

Output Format:

1. Data File (output.txt)

Format: A binary file that is exact byte-for-byte copy of the input.txt file.

Purpose: This file's integrity is the ultimate measure of a protocol's success. A perfect copy proves that the chosen protocol successfully and reliably transferred the original data despite any simulated network problems.

2. Terminal Log Output

Format: Human-readable text printed to the console in both the sender and receiver windows.

Purpose: To provide a real-time, step-by-step trace of the simulation. This log shows events like frame transmissions, timeouts, retransmissions, ACK receipts, and error detections, making it the primary tool for observing and debugging the protocol's behaviour.

3. Efficiency Report

Format: A structured, text-based summary printed to the sender's terminal at the end of each simulation.

Purpose: To provide a quantitative summary of the protocol's performance. It includes metrics like total time elapsed, the number of retransmissions, and the effective data throughput in Kilobits per second (Kbps).

Implementation

This program simulates three flow control protocols: Stop-and-Wait, Go-Back-N, and Selective Repeat. It sends a file over a simulated unreliable channel, allowing a user to compare each protocol's performance.

config.py

This file centralizes all simulation settings. It controls network addresses, payload size, protocol parameters like window size and timeouts, and the probabilities for simulated network errors, losses, and delays.

```
#    Network configuration for UDP sockets

SENDER_IP = "127.0.0.1"
RECEIVER_IP = "127.0.0.1"
SENDER_PORT = 8080
RECEIVER_PORT = 8081

INPUT_FILE = "input.txt"
OUTPUT_FILE = "output.txt"

PAYLOAD_SIZE = 128 # In BYTES, must be between 46 and 1500 per the document.
HEADER_SIZE = 15      # 6 (src) + 6 (dst) + 2 (len) + 1 (seq) bytes
CRC_SIZE = 4
CONTROL_FRAME_SIZE = 6

WINDOW_SIZE_N = 4      # Window size for Go-Back-N and Selective Repeat
SEQ_NUM_BITS = 8        # The sequence number is 1 byte (8 bits)
SEQ_NUM_RANGE = 2**SEQ_NUM_BITS # This gives a range of 0-255
TIMEOUT_INTERVAL = 2.0 # seconds

# Channel simulation
ERROR_PROBABILITY = 0.1 # P(bit error) LOSS_PROBABILITY = 0.1      #
P(frame lost)
DELAY_PROBABILITY = 0.1 # P(frame delayed)

#    Delay Settings
MIN_DELAY = 1.0
MAX_DELAY = 3.0
```

crc.py

This module provides functions for error detection. It calculates a Cyclic Redundancy Check (CRC) for an outgoing frame and verifies the integrity of an incoming frame using the same algorithm.

```
import zlib
```

```
def calculate_crc(data: bytes) -> bytes:
    crc_value = zlib.crc32(data)
    return crc_value.to_bytes(4, byteorder='big', signed=False)
```

```
def verify_crc(data_with_crc: bytes) -> bool:
    """Verifies if the data has a valid CRC. Assumes last 4 bytes are the CRC."""
    if len(data_with_crc) < 4:
        return False
```

```
    data = data_with_crc[:-4]
    received_crc_bytes = data_with_crc[-4:]
```

```
    calculated_crc_bytes = calculate_crc(data)
```

```
    return received_crc_bytes == calculated_crc_bytes
```

frame_handler.py

Defines the structure of a data frame and acknowledgment (ACK). This module handles the packing of data into a byte stream for sending and unpacking received bytes back into a frame object.

```
# frame_handler.py
import struct
import config
import crc
```

```
HEADER_FORMAT = "! 6s 6s H B"
```

```
KIND_ACK = 0x01
```

```
KIND_NACK = 0x02
```

```
CONTROL_FRAME_FORMAT = "! B B" # Format is: Kind (1 byte), Seq Num (1 byte)
```

```
class Frame:
```

```
    """Represents a single data frame for transmission."""
```

```
    def __init__(self, seq_no: int, payload: bytes, src_addr: bytes = b'\xAA'*6,
                 dest_addr: bytes = b'\xBB'*6, crc: bytes | None = None):
```

```

self.seq_no = seq_no self.payload =
payload self.src_addr = src_addr
self.dest_addr = dest_addr self.length
= len(payload) self.crc = crc

def pack(self) -> bytes: header = struct.pack(HEADER_FORMAT, self.src_addr,
self.dest_addr,
self.length, self.seq_no) data_to_checksum = header +
self.payload crc_bytes =
crc.calculate_crc(data_to_checksum) return
data_to_checksum + crc_bytes

@classmethod def unpack(cls,
raw_bytes: bytes):
    if not crc.verify_crc(raw_bytes): return None
    header = raw_bytes[:config.HEADER_SIZE] src_addr, dest_addr, length, seq_no
    = struct.unpack(HEADER_FORMAT,
header)
    payload = raw_bytes[config.HEADER_SIZE:-config.CRC_SIZE]
    if length != len(payload): return None crc_from_trailer =
    raw_bytes[-config.CRC_SIZE:]
    return cls(seq_no, payload, src_addr, dest_addr, crc=crc_from_trailer)
def create_control_frame(seq_no: int, kind: int) -> bytes:
    """Creates a control frame (ACK or NACK) for the given sequence number."""
    control_data = struct.pack(CONTROL_FRAME_FORMAT, kind, seq_no) crc_bytes =
    crc.calculate_crc(control_data) return control_data + crc_bytes

def unpack_control_frame(control_bytes: bytes) -> tuple[int, int] | None:
    """Unpacks a control frame. Returns (kind, sequence number) or None if
    corrupt.""" if not crc.verify_crc(control_bytes): return None
    try:
        kind, seq_no = struct.unpack(CONTROL_FRAME_FORMAT,
control_bytes[-config.CRC_SIZE:]) return
        kind, seq_no
    except struct.error:
        return None

```

channel.py

Simulates an unreliable network link between the sender and receiver. It intercepts all transmitted frames and, based on probabilities from config.py, can introduce bit errors, packet loss, or random delays.

```

import random
import time
import config

```



```

def _inject_error(frame_bytes: bytearray) -> bytearray:
    """Injects a single-bit error into the frame.""" if not
    frame_bytes: return frame_bytes
    length_bits = len(frame_bytes) * 8 pos =
    random.randint(0, length_bits - 1)
    byte_index = pos // 8 bit_in_byte = pos % 8
    frame_bytes[byte_index] ^= (1 << bit_in_byte)
    print(f'NETWORK: Injected a bit error at bit position
    {pos}.') return frame_bytes
def transmit(sock, receiver_address, frame_bytes: bytes):
    """Transmits a frame through the simulated channel.""" rand_val =
    random.random()
    if rand_val < config.LOSS_PROBABILITY: print(f'NETWORK: Packet
    lost!') return

    elif rand_val < config.LOSS_PROBABILITY +
    config.ERROR_PROBABILITY:
        print(f'NETWORK: Packet corrupted!') corrupted_frame =
        _inject_error(bytearray(frame_bytes))
        sock.sendto(corrupted_frame, receiver_address)
    elif rand_val < config.LOSS_PROBABILITY +
    config.ERROR_PROBABILITY + config.DELAY_PROBABILITY: delay =
    random.uniform(config.MIN_DELAY, config.MAX_DELAY)
    print(f'NETWORK: Delaying packet for {delay:.2f} seconds...')
    time.sleep(delay)
    sock.sendto(frame_bytes, receiver_address) else:
    sock.sendto(frame_bytes, receiver_address)

```

sender.py

The main client program. It reads a file, splits it into chunks, and implements the logic for Stop-and-Wait, Go-Back-N, and Selective Repeat to reliably transmit the data to the receiver.

```

import socket
import time
import config
from frame_handler import Frame, unpack_control_frame, KIND_ACK,
KIND_NACK
import channel
total_retransmissions = 0

```

```

def print_efficiency_report(total_frames, start_time, end_time, payload_size):

```

```

        global total_retransmissions        total_time_sec =
end_time - start_time print("\n" + "-"*27); print("--- Efficiency
Report ---")        print(f"Total data frames to send:
{total_frames}"); print(f"Total retransmissions:
        {total_retransmissions}")
        print(f"Total time elapsed:    {total_time_sec:.2f} seconds")        if
total_time_sec > 0:
        total_data_bytes = total_frames * payload_size throughput_kbps =
        (total_data_bytes * 8) / total_time_sec / 1000 print(f"Effective
        throughput: {throughput_kbps:.2f} Kbps")
print("-" * 27)

def    run_stop_and_wait(sock, receiver_address, chunks):
    global total_retransmissions; total_chunks = len(chunks); seq_no =
0; chunk_index = 0        while chunk_index < total_chunks: frame =
Frame(seq_no, chunks[chunk_index]); packed_frame
= frame.pack(); ack_received = False        while
not ack_received:
        print(f"S&W: Sending chunk {chunk_index + 1}/{total_chunks} with
seq_no    {seq_no}...")
        channel.transmit(sock, receiver_address, packed_frame)
        try: sock.settimeout(config.TIMEOUT_INTERVAL); control_bytes, _ =
sock.recvfrom(1024)
        control_data = unpack_control_frame(control_bytes)        if
control_data and control_data[0] == KIND_ACK:
            ack_seq_no = control_data[1] if
            ack_seq_no == seq_no:

                print(f"S&W: Received correct ACK for seq_no
{seq_no}."); ack_received = True
            else:    print(f"S&W: Received wrong ACK (got {ack_seq_no}).
Ignoring.")
                except socket.timeout:
                    print(f"S&W: Timeout! Retransmitting..."); total_retransmissions += 1
                    seq_no = (seq_no + 1) % 2; chunk_index += 1 sock.sendto(b'END', receiver_address)

def    run_go_back_n(sock, receiver_address, chunks):
    global total_retransmissions; total_chunks = len(chunks); window_size =
config.WINDOW_SIZE_N

```

```

base = 0; next_seq_num = 0; sent_frames = {}
while base < total_chunks:
    while next_seq_num < base + window_size and next_seq_num < total_chunks:
        frame_seq_num = next_seq_num % config.SEQ_NUM_RANGE

        frame = Frame(frame_seq_num, chunks[next_seq_num]); packed_frame
= frame.pack()
        sent_frames[frame_seq_num] = packed_frame print(f"GBN: Sending chunk
        {next_seq_num + 1}/{total_chunks} with
seq_no      {frame_seq_num}..." )      channel.transmit(sock, receiver_address,
packed_frame); next_seq_num
+=      1
        try:
            sock.settimeout(config.TIMEOUT_INTERVAL); control_bytes, _ =
sock.recvfrom(1024)
            control_data = unpack_control_frame(control_bytes)      if
control_data and control_data[0] == KIND_ACK:
                ack_num = control_data[1] if base <
                ack_num <= next_seq_num:

                    print(f"GBN: Received cumulative ACK {ack_num}. Window
slides.");      base = ack_num
            except socket.timeout:
                print(f"\nGBN: Timeout! Re-sending window from base {base}.\n")
                total_retransmissions += (next_seq_num - base)      for i in range(base,
next_seq_num):
                    channel.transmit(sock, receiver_address, sent_frames[i % config.SEQ_NUM_RANGE])

            sock.sendto(b'END', receiver_address)

def run_selective_repeat(sock, receiver_address, chunks):

    global total_retransmissions; total_chunks = len(chunks); window_size =
config.WINDOW_SIZE_N

    base = 0; next_seq_num = 0; sent_frames = {}; acked_frames = [False]
* total_chunks; send_timestamps = {}      while
base < total_chunks:

        while next_seq_num < base + window_size and next_seq_num < total_chunks:
            frame_seq_num = next_seq_num % config.SEQ_NUM_RANGE

            frame = Frame(frame_seq_num, chunks[next_seq_num]); packed_frame

```

```

= frame.pack() sent_frames[frame_seq_num] =
    packed_frame print(f"SR: Sending chunk
        {next_seq_num + 1}/{total_chunks}
with seq_no {frame_seq_num}...")

    channel.transmit(sock, receiver_address, packed_frame);
send_timestamps[frame_seq_num] = time.time(); next_seq_num += 1
    try:
        sock.settimeout(0.1);
control_bytes, _ = sock.recvfrom(1024)
control_data = unpack_control_frame(control_bytes)    if control_data:
    kind, seq_num = control_data        if kind == KIND_NACK: print(f"SR: Received
NACK for frame {seq_num}! Retransmitting
    immediately...")
        if seq_num in sent_frames:
            channel.transmit(sock, receiver_address, sent_frames[seq_num])
            send_timestamps[seq_num] = time.time(); total_retransmissions
+= 1

        elif kind == KIND_ACK:
            acknowledged_chunk_index = -1
            for i in range(base, next_seq_num):
                if (i % config.SEQ_NUM_RANGE) == seq_num:

                    acknowledged_chunk_index = i; break

            if acknowledged_chunk_index != -1 and
not acknowledged_frames[acknowledged_chunk_index]:

print(f"SR: Received ACK for frame {seq_num} (chunk { acknowledged_chunk_index + 1}).")
        acknowledged_frames[acknowledged_chunk_index] = True        while base < total_chunks and
        acknowledged_frames[base]: base += 1    print(f"SR: Window base is now at chunk {base +
1}).")        except socket.timeout: pass    current_time = time.time()    for i in
range(base, next_seq_num):

        frame_seq_num_to_check = i % config.SEQ_NUM_RANGE
        if not acknowledged_frames[i] and (current_time -

            send_timestamps[frame_seq_num_to_check] > config.TIMEOUT_INTERVAL):
                print(f"SR: Timeout for frame {frame_seq_num_to_check}! Resending chunk {i + 1}...")

            channel.transmit(sock, receiver_address,
sent_frames[frame_seq_num_to_check])

                send_timestamps[frame_seq_num_to_check] = current_time;
total_retransmissions += 1

```

```

sock.sendto(b'END', receiver_address)

def main():
    global total_retransmissions; sock = socket.socket(socket.AF_INET,
socket.SOCK_DGRAM)
    receiver_address = (config.RECEIVER_IP, config.RECEIVER_PORT)
    try:
        with open(config.INPUT_FILE, "rb") as f: data = f.read()
    except
FileNotFoundError:
        print(f"Error: Input file '{config.INPUT_FILE}' not found."); return chunks =
[data[i:i + config.PAYLOAD_SIZE] for i in range(0, len(data),
config.PAYLOAD_SIZE)]
        while True:
            print("\n" + "="*40)
            choice = input("Choose a protocol:\n1. Stop-and-Wait\n2. Go-
Back-N\n3. Selective Repeat (with NACK)\n4. Quit\nEnter choice: ")
            print("="*40)
            if choice in ['1', '2', '3']:
                total_retransmissions = 0; start_time = time.time()
                sock.sendto(f"START:{choice}".encode(), receiver_address)
                if choice == '1':
                    run_stop_and_wait(sock, receiver_address, chunks)
                elif choice == '2':
                    run_go_back_n(sock, receiver_address, chunks)
                elif choice == '3':
                    run_selective_repeat(sock, receiver_address, chunks)
                end_time = time.time()
                print_efficiency_report(len(chunks), start_time, end_time,
config.PAYLOAD_SIZE)

            print(f"\n--- Protocol {choice} simulation finished. Returning to menu. --- ")

            elif choice == '4':
                print("Quit selected."); break
            else:
                print("Invalid choice.")
                print("\nSender session ended.
Notifying receiver to quit.")
                sock.sendto(b'QUIT', receiver_address);
                sock.close()

if __name__ == "__main__":
    main()

```

receiver.py

The main server program. It listens for incoming data, implements the receiving-end logic for all three ARQ protocols, validates frames, writes correct data to a file, and sends acknowledgments.

```

import socket
import config

from frame_handler import Frame, create_control_frame, KIND_ACK,
KIND_NACK

def display_header_details(frame):
    """A helper function to print formatted header and trailer info."""
    print("\n--- Frame Header & Trailer Info ---")
    src_addr_str = ':'.join(f'{b:02X}' for b in frame.src_addr)
    dest_addr_str = ':'.join(f'{b:02X}' for b in frame.dest_addr)
    print(f"Source Address: {src_addr_str}"); print(f"Destination Address: {dest_addr_str}")
    if frame.crc: print(f"Trailer (FCS/CRC): 0x{frame.crc.hex().upper()}"); print("-----")

def run_stop_and_wait_receiver(sock, sender_address):
    print("--- Stop-and-Wait Receiver Activated ---"); expected_seq_no = 0;
    header_displayed = False
    with open(config.OUTPUT_FILE, "wb") as f:
        while True:
            try:
                raw_bytes, _ = sock.recvfrom(2048)
                if raw_bytes == b'END': return
                frame = Frame.unpack(raw_bytes)
                if frame:
                    if not header_displayed: display_header_details(frame);
                    header_displayed = True
                    print(f"\nReceiver: Received {len(raw_bytes)} bytes.")
                    if frame.seq_no == expected_seq_no:
                        print(f"S&W: Frame {frame.seq_no} is correct. Writing to file.")
                        f.write(frame.payload); f.flush();

            sock.sendto(create_control_frame(expected_seq_no, KIND_ACK), sender_address)

            print(f"S&W: Sent ACK for seq_no {expected_seq_no}.");
            expected_seq_no = (expected_seq_no + 1) % 2
            else: print(f"S&W: Duplicate frame {frame.seq_no} received. Discarding.")
            sock.sendto(create_control_frame(frame.seq_no, KIND_ACK), sender_address); print(f"S&W: Re-sent ACK for seq_no {frame.seq_no}.")
            else:

```

```

        print(f'\nReceiver: Received {len(raw_bytes)} bytes.");
print(f'S&W:      Corrupt frame received. Discarding.")      except
Exception as e: print(f'An error occurred during S&W session: {e}");
return

def run_go_back_n_receiver(sock, sender_address):

    print("--- Go-Back-N Receiver Activated ---"); expected_seq_num =
0; header_displayed = False    with
open(config.OUTPUT_FILE, "wb") as f:
    while True:
        try:
            raw_bytes, _ = sock.recvfrom(2048)    if
raw_bytes == b'END': return    frame =
Frame.unpack(raw_bytes) if frame:

                if not header_displayed: display_header_details(frame);
header_displayed        = True        print(f'\nReceiver: Received
{len(raw_bytes)} bytes.")        if frame.seq_no ==
expected_seq_num:
            print(f'GBN: Frame {frame.seq_no} is correct. Writing to file.")
            f.write(frame.payload); f.flush(); expected_seq_num =
(expected_seq_num + 1) % config.SEQ_NUM_RANGE
            sock.sendto(create_control_frame(expected_seq_num,
KIND_ACK), sender_address); print(f'GBN: Sent cumulative ACK
{expected_seq_num}.")
        else:
            print(f'GBN: Out-of-order frame {frame.seq_no} (expecting
{expected_seq_num}). Discarding.")

            sock.sendto(create_control_frame(expected_seq_num,
KIND_ACK), sender_address); print(f'GBN: Re-sent cumulative ACK
{expected_seq_num}.")

        else:
            print(f'\nReceiver: Received {len(raw_bytes)} bytes."); print(f'GBN: Corrupt
frame received. Discarding.")

            except Exception as e: print(f'An error occurred during GBN
session: {e}"); return

```

```

def run_selective_repeat_receiver(sock, sender_address):

    print("--- Selective Repeat Receiver Activated (with NACKs) ---");
    window_size      = config.WINDOW_SIZE_N      base = 0; buffer = {};
    header_displayed = False; nacks_sent = set()    with
    open(config.OUTPUT_FILE, "wb") as f:
        while True:
            try:
                raw_bytes, _ = sock.recvfrom(2048)    if raw_bytes == b'END': return
                frame = Frame.unpack(raw_bytes)    if not frame:
                print(f"\nReceiver: Received {len(raw_bytes)} bytes."); print(f"SR:
Corrupt    frame received. Discarding."); continue if not header_displayed:
                display_header_details(frame);        header_displayed = True
                print(f"\nReceiver: Received {len(raw_bytes)} bytes."); print(f"SR:
Received frame with seq_no {frame.seq_no}.")

                base_seq = base % config.SEQ_NUM_RANGE
                if frame.seq_no != base_seq:    if base_seq not in
                nacks_sent:

                    print(f"SR: Gap detected! Expecting {base_seq},
                    got {frame.seq_no}. Sending NACK for {base_seq}.")

                    sock.sendto(create_control_frame(base_seq,
                    KIND_NACK), sender_address); nacks_sent.add(base_seq)

                is_in_rcv_window = False for i
                in range(window_size):

                    if frame.seq_no == (base + i) % config.SEQ_NUM_RANGE:
                        is_in_rcv_window = True; acked_chunk_num = base + i if
                        acked_chunk_num not in buffer:

                            buffer[acked_chunk_num] = frame.payload
                            print(f"SR: Frame {frame.seq_no} accepted and buffered
                            (chunk {acked_chunk_num+1}).")

                            sock.sendto(create_control_frame(frame.seq_no,
                            KIND_ACK), sender_address); print(f"SR: Sent ACK for {frame.seq_no}.") break

                            if not is_in_rcv_window: print(f"SR: Frame {frame.seq_no} is
                            outside current window [base={base}]. Discarding.")

```



```

        while base in buffer:
            payload = buffer.pop(base); f.write(payload); f.flush()
            if base in
nacks_sent: nacks_sent.remove(base) print(f"SR: Delivered chunk {base + 1} from
buffer. Window base is
now {base + 1}."); base += 1
except Exception as e:
print(f"An error occurred during SR session: {e}"); return

```

```

def main():

```

```

    sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
    sock.bind((config.RECEIVER_IP, config.RECEIVER_PORT)) print(f"Receiver:
Listening on

```

```

{config.RECEIVER_IP}:{config.RECEIVER_PORT}")
    while
True:

```

```

        print("\n--- Waiting for Sender to start a session... ---")

```

```

        try:

```

```

            raw_bytes, sender_address = sock.recvfrom(2048)
            message =
raw_bytes.decode()
            if message.startswith("START:"):
                choice =
message.split(':')[1]
            if choice == '1': run_stop_and_wait_receiver(sock,
sender_address)
            elif choice == '2': run_go_back_n_receiver(sock,
sender_address)
            elif choice == '3': run_selective_repeat_receiver(sock,
sender_address)
            print("--- Session finished. Returning to wait state. ---")

```

```

            elif raw_bytes == b'QUIT': print("Receiver: Quit signal received. Shutting
down.");

```

```

            break
        else:

```

```

                print(f"Receiver: Waiting for a START signal, but received unexpected
data.")
            except

```

```

Exception as e:

```

```

    print(f"Main listener error: {e}")
    sock.close()

```

```

if __name__ == "__main__":

```

```

    main()

```

Results

The program uses three primary performance metrics for evaluation: Total Retransmissions, Total Time Elapsed, and Effective Throughput.

Performance Metrics:

These metrics are calculated and displayed in the efficiency report at the end of each simulation to help compare the protocols.

Total Retransmissions: This is a count of every time a frame is resent due to a timeout. It's a direct measure of a protocol's efficiency in an unreliable network; a lower number indicates better performance as less redundant work was done.

Total Time Elapsed: This measures the full duration of the file transfer, from the moment the first frame is sent until the simulation for that protocol completes. It's measured in seconds, and a shorter time is better.

Effective Throughput: This is a calculated metric that measures the actual data transfer rate in Kilobits per second (Kbps). It's derived from the total amount of useful data (the original file size) divided by the total time elapsed. A higher throughput value signifies a more efficient transfer.

Analysis

The simulation demonstrates that while all three protocols - Stop-and-Wait, Go-Back-N, and Selective Repeat ensure correct data transfer, their performance varies significantly under different network conditions. Go-Back-N and Selective Repeat offer much higher throughput in ideal networks, while Selective Repeat is the most resilient to high error rates.

Correctness and Reliability

The most important result across all test cases is that the program is correct. For every simulation, regardless of the protocol chosen or the severity of network errors, the final output.txt is a perfect, byte-for-byte copy of input.txt. This confirms that all three ARQ implementations successfully achieve their primary goal: providing a reliable data transfer service over an unreliable channel.

Discussion of Results for Test Cases

The performance of each protocol, as measured by the efficiency report, aligns with theoretical expectations.

Stop-and-Wait (S&W): On a perfect network, S&W is functional but slow. Its "one frame at a time" nature leads to the lowest throughput as the channel is idle most of the time. Under error conditions, its performance degrades severely because a single lost frame halts all progress for the full timeout duration.

Go-Back-N (GBN): On a perfect network, GBN is significantly faster than S&W. Its sliding window allows it to "pipeline" frames and use the channel much more efficiently, resulting in high throughput. However, its performance drops sharply as error rates increase. Because it must retransmit the *entire window* for a single packet loss, it wastes bandwidth and time on unnecessary retransmissions.

Selective Repeat (SR): This protocol offers the best of both worlds. On a perfect network, its performance is as high as GBN's. Under error conditions, it is the most efficient. By only retransmitting the *specific frames* that were lost and buffering out-of-order frames, it avoids the wasteful retransmissions of GBN, maintaining the highest throughput when the network is unreliable.

Impact of Errors and Delays

Errors and delays universally decrease performance for all protocols by triggering timeouts and retransmissions. The key difference is the cost of recovery:

S&W and SR Cost: 1 retransmitted frame per error.

GBN Cost: N retransmitted frames per error (where N is the window size).

This difference in cost is why GBN's efficiency falls off much more dramatically than SR's as network conditions worsen.

Known Bugs and Limitations

The program also has a few design limitations:

Simulated Timers in SR: The sender simulates individual timers for frames in the SR window using timestamps. A true implementation would use concurrent timers, but this approach is a reasonable and common simplification.

Fixed Timeout Value: The simulation uses a fixed TIMEOUT_INTERVAL. A more advanced implementation would dynamically calculate the Round-Trip Time (RTT) and adjust the timeout value accordingly.